

P?S Enhanced PDP-8 Simulator

User Guide

The logo consists of the characters 'P', '?', and 'S' in a white, serif font, centered on a solid blue rectangular background. The characters are large and bold, with the question mark being slightly smaller than the letters.

P?S

Name: P?S Enhanced PDP-8 Simulator User Guide
Document: PEPS User Guide.rtf
File[s]: PEPS User Guide.PDF
P?S Index: O>R,ODOR0037TRDQ0037FTHCD
Security Level: 2
Location: NY
Date: 15-Dec-2016
Medium: PDF

Private Prerelease October 2016
1st Public Release December 2016

P?S Enhanced PDP-8 Simulator

User Guide

Copyright © 1990-2016 »CLA Systems

Contact:
CLASystems@gmail.com

CONTENTS

CHAPTER 1 INTRODUCTION

- 1.1 General Description
- 1.2 General Usage
- 1.3 Supported PDP-8 Operating Systems
- 1.4 Using the SIMH simulator for program development
 - 1.41 Using the SIMH simulator for basic PDP-8 programming
 - 1.42 Using the full PEPS environment for PDP-8 programming
 - 1.43 Host editing and development of PDP-8 source files
- 1.50 Operating the PEPS from the Command directory
- 1.60 Description of individual commands within the Command window
- 1.61 Supported Operating Systems
- 1.62 Generic file handling and related commands
- 1.63 Custom commands for individual file development
- 1.64 Special-purpose utility commands
- 1.65 Miscellaneous commands

This page intentionally left blank.

CHAPTER 1

INTRODUCTION

The *P?S Enhanced PDP-8 Simulator* (PEPS) is a software implementation of the 12-bit PDP-8 computer including most of the peripheral devices often available within specific hardware configurations. Many of the PEPS package components were created by members of the P?S^[1].

The overall simulation is controlled by specific initialization and configuration files passed to the basic "engine" of the simulator. This internal component is generally known as the SIMH^[2] simulator for the *Digital Equipment Corporation* (DEC) PDP-8 family of computers.

Many of the available SIMH configuration parameters concern simulation of hardware devices not required by the operation of the supported single-user operating systems provided in the PEPS package. For those users seeking additional documentation, links are provided to websites that may supply additional information on certain obscure features of SIMH; however, most users of PEPS will not be concerned unless intending to develop software expressly targeting these devices.

While the PEPS package can be used to simulate minimal PDP-8 configurations, emphasis is given to larger systems that include various single-user storage-based PDP-8 operating systems. This allows execution of more complex programs, some of which are specific to the operating system environment chosen.

If desired, new programs can be created using a variety of development techniques. In the PEPS environment, program development focuses on using many facilities available in the host Microsoft Windows environment. Depending on the user's needs, this can include creation of output files that can be viewed in the host environment; in specific cases, various forms of post-processing can be applied for the purpose of creating output suitable for printing on Windows-compliant printers in a variety of output formats. This could also include creation of fully formatted listing files of certain older programs that were never available previously (other than perhaps in the form of primitive and/or badly copied paper listings with many limitations inhibiting any meaningful usage).

This User Guide, together with the documents referenced in various appendices, provides information on the installation, maintenance and operation of the PEPS package.

The PEPS package is designed to work on any release of Microsoft Windows XP, Windows Vista, Windows 7, Windows 8 or Windows 10. It is generally distributed as a self-extracting archive capable of being installed on any permanently lettered logical drive. Due to internal restrictions, the PEPS package cannot be located on network shares or removable drives where the logical drive letter is subject to change. Appendix B details the installation process, including the ability to create custom installation packages should the user create any additional PEPS features beyond the standard release.

Note: For illustration purposes only, the PEPS package elements will be described throughout this document as if installed on Windows logical drive P (P:). Any permanently lettered logical device may be used; however, it is recommended the PEPS package be installed on a dedicated drive to facilitate backup of working copies of the package.

While not widely known, the use of multiple logical drives in Windows is actually encouraged by Microsoft. This is best accomplished using various third-party hard drive partitioning programs; however, certain Windows releases include Administrative Tools capable of performing a rudimentary form of partitioning that may be sufficient for many users. Appendix C documents several third-party partitioning programs that are compatible with all systems that support the PEPS package. These programs include additional features that may be of use to advanced users beyond the scope of placement of the PEPS package.

Certain auxiliary utility programs compatible with Windows are optionally required depending on the user's needs; Appendix C also documents these additional programs. Among these utilities are various text editors known to be compatible with the PEPS package and post-processing programs to facilitate viewing or printing text files created by PEPS (such as assembly listings of user programs).

Most of the PEPS documentation files (including this User Guide) are distributed in *Portable Document Format* (PDF). As such, a compatible document reading program (such as *Adobe Reader*) should be installed prior to the first usage of the PEPS package.

1.1 General Description

The PEPS package provides a functional simulation of a PDP-8 system with various peripherals. When compared to real-world hardware of various PDP-8 systems, the configuration is considered to be a large system. This provides much of the performance of the original PDP-8 systems while actually running on a Windows-based personal computer.

As typically configured, the following 12-bit resources are generally available within the PEPS environment:

- a) PDP-8/E CPU [generally known as KK8F as found in the PDP-8/E, PDP-8/F, PDP-8/M and the PDP-8/A-620 series].
- b) 32K of main memory.
- c) *Extended Arithmetic Element* (EAE). Note: Due to the presence of EAE, the corresponding PDP-8/A model is known as the PDP-8/A-625.

Note: By adjusting certain parameters within the configuration files, the specific model of PDP-8 and availability of EAE can be changed to support certain earlier models. Appendix Q contains more information on SIMH parameters which may be of interest to advanced users.

- d) Standard PDP-8 device 03/04 console terminal with full support of DEC standard horizontal tab characters. Advanced console terminal-handling software can depend on proper display of the console output showing horizontal tabs aligned every eight spaces, as would typically be the case on newer DEC terminals such as the VT100. However, in most respects, the console display resembles the output of a Teletype Model 33 using pale yellow roll paper and bold black ink ribbon.

Note: For operating systems with known defects in console terminal handling such as OS/8^[3], the console simulation can be restricted forcing upper-case input and output, as well as suppression of unintended output of certain control characters.

- e) Device 66 standard line printer (such as LP08). Note: For certain configurations this can be deliberately disabled as necessary.
- f) PC04 High-Speed Reader (device 01) and Punch (device 02). Note: Individual file transfer from the PEPS host environment into the SIMH environment can only be achieved using the high-speed reader. Individual file transfer from the SIMH environment to the PEPS host environment can be accomplished using either the device 66 line printer or the high-speed punch.
- g) TC08 (or TC01) DEctape Controller with up to eight DEctape drive units numbered 0 through 7. Note: Each logical drive must be configured by being associated with an active compatible device file that represents the contents of the DEctape mounted on the particular drive. If the associated file doesn't exist, it will automatically be created when attempting to write to the drive. Any non-configured drive will return a SELECT error if read access is attempted.
- h) Primary (device 77) TD8E Controller with up to two DEctape drives with the same file considerations as in g) above. Note: The TD8E controller and the TC08 controller are mutually exclusive.
- i) RK8E (device 74) Cartridge Disk Controller with up to four RK05 drives. As in the case of DEctape drives, should the associated drive file not exist, it will automatically be created.
- j) Primary (device 75) RX8E Controller with up to two RX01 drives. As in the case of RK05 drives, should the associated drive file not exist, it will automatically be created.

Additional devices are available depending on the specific configuration of the SIMH simulator. Appendix D refers to these devices; however, specific software drivers or additional utility programs may be required for functional support.

Note: A limitation of the SIMH environment is that dismountable devices such as DECTapes, RK05 and RX01 drives cannot be loaded or unloaded during SIMH sessions; a design goal of the PEPS package is to allow advanced users the ability to rapidly modify the specific configuration followed by a restart of the SIMH simulation process to mitigate this issue. Wherever possible, the various components of the simulator are maintained in separate directories to avoid needless confusion. Special emphasis is given to accessing the configuration files that determine the specific simulation. Any number of configurations can be available on a given system; however only one such configuration can be run at any given time. This allows maximum flexibility in terms of mounted device specifics and operating system configurations from a dedicated directory.

1.2 General Usage

Note: The PDP-8 supports several different file structures, all referred to in the 12-bit world as *directories*. In Windows, the analogous structures are often referred to as *folders*. Within this User Guide, the terms will be used interchangeably when referring to the file structures of Microsoft Windows.

Using the PEPS package typically starts by calling up the PEPS Command directory to the Windows desktop; this serves as the "command center" for most PEPS operations. From this folder, many functions and utilities can be initiated. This includes bringing up the SIMH PDP-8 simulator booting to one of several supported operating system configurations and various file-related functions to allow file editing and file maintenance in the host Windows environment.

The user must have a general understanding of Microsoft Windows to make use of many of the available features of the PEPS package. Knowledge of the PDP-8 operating systems is essential to use the various facilities provided and, if desired, help to draw comparisons between the supported operating systems. Appendix A includes references to documents for the various supported PDP-8 operating systems which can aid in their usage, some of which are provided in the complete PEPS distribution.

The Command directory is described in greater detail in section 1.50 below.

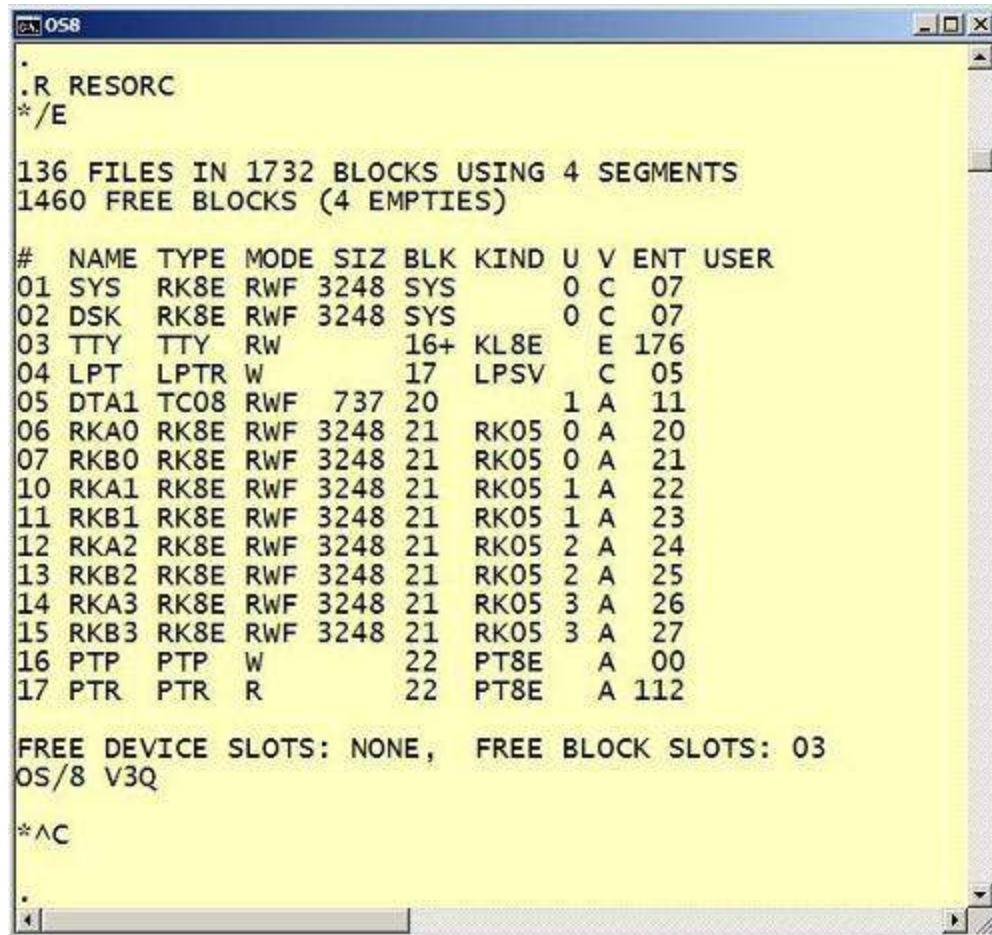
Note: The PEPS package includes desktop icons/shortcuts to both the Command directory and the additional directory group which supports it. All users generally start in the Command directory to initiate most operations while the underlying directory group is only accessed by advanced users and/or developers of the PEPS package itself. It is strongly recommended that both of these desktop shortcuts be installed on all systems; this will allow a smooth transition as the user gains familiarity with the full complement of implemented features (including available customization techniques).

1.3 Supported PDP-8 Operating Systems

As of this writing, various configurations of two different PDP-8 operating systems are supported.

1) OS/8 Version V3D (V3Q)

Figure 1 shows console output specific to the OS/8 *RESORC* utility used to display the current configuration of supported devices.



```
.R RESORC
*/E

136 FILES IN 1732 BLOCKS USING 4 SEGMENTS
1460 FREE BLOCKS (4 EMPTIES)

#  NAME TYPE MODE SIZ BLK KIND U V ENT USER
01 SYS  RK8E RWF 3248 SYS      0 C  07
02 DSK  RK8E RWF 3248 SYS      0 C  07
03 TTY   TTY  RW      16+ KL8E   E 176
04 LPT  LPTR  W      17  LPSV   C  05
05 DTA1 TC08 RWF  737 20      1 A  11
06 RKA0 RK8E RWF 3248 21  RK05  0 A  20
07 RKB0 RK8E RWF 3248 21  RK05  0 A  21
10 RKA1 RK8E RWF 3248 21  RK05  1 A  22
11 RKB1 RK8E RWF 3248 21  RK05  1 A  23
12 RKA2 RK8E RWF 3248 21  RK05  2 A  24
13 RKB2 RK8E RWF 3248 21  RK05  2 A  25
14 RKA3 RK8E RWF 3248 21  RK05  3 A  26
15 RKB3 RK8E RWF 3248 21  RK05  3 A  27
16 PTP  PTP   W      22  PT8E   A  00
17 PTR  PTR   R      22  PT8E   A 112

FREE DEVICE SLOTS: NONE,  FREE BLOCK SLOTS: 03
OS/8 V3Q

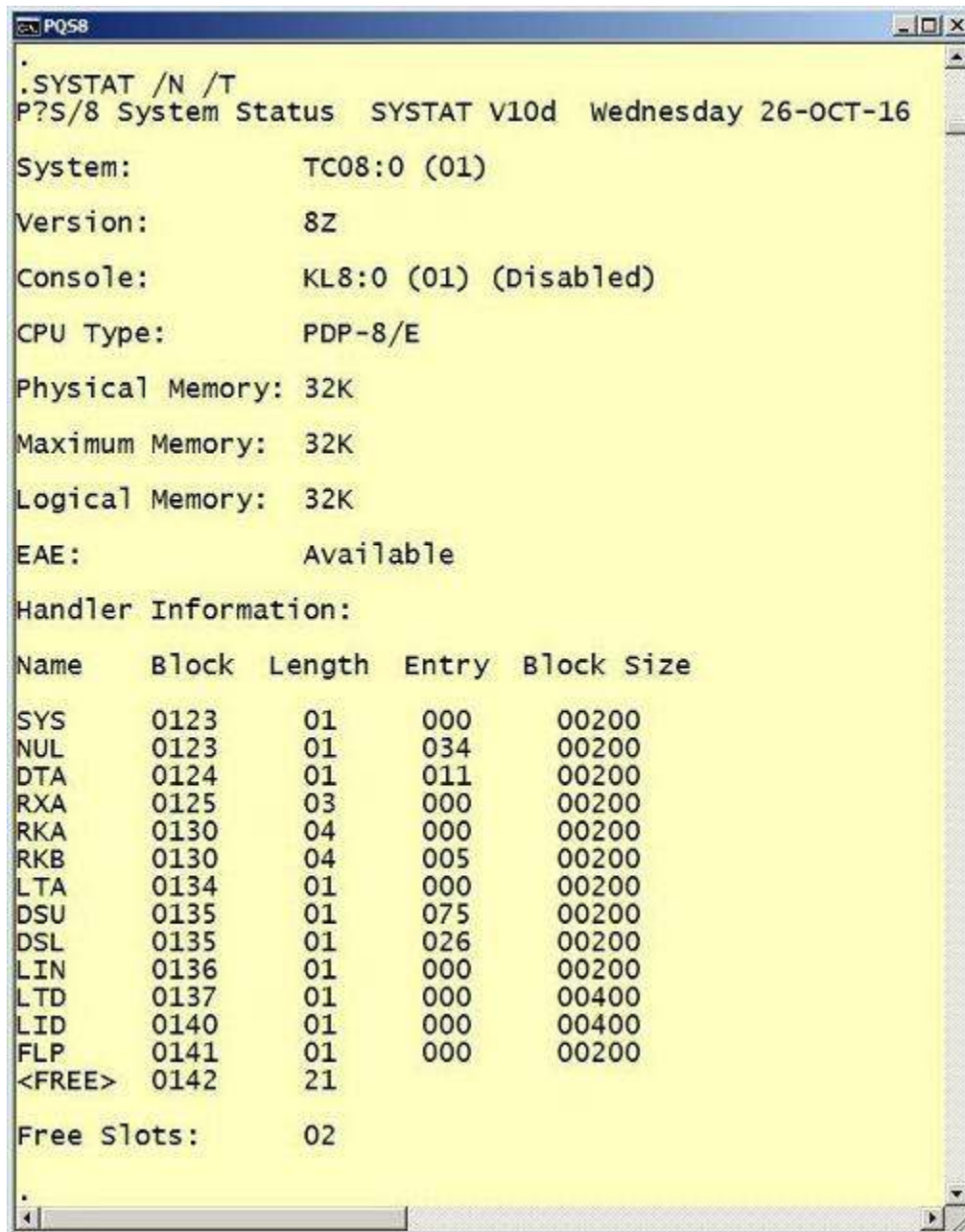
*AC
```

Figure 1. OS/8 Supported Resources

Note: The exact configuration for any OS/8 system can be modified using the OS/8 *BUILD* program as long as all devices are supported by the SIMH simulator. Several alternate configuration copies of *BUILD* are provided in the PEPS implementation of OS/8 to allow creation of the most common variations.

2) P?S/8^[4] Version 8Z

Figure 2 shows console output specific to the P?S/8 *SYSTAT* utility used to display the current configuration of supported devices.



```
.SYSTAT /N /T
P?S/8 System Status  SYSTAT V10d  Wednesday 26-OCT-16

System:          TC08:0 (01)
Version:         8Z
Console:         KL8:0 (01) (Disabled)
CPU Type:        PDP-8/E
Physical Memory: 32K
Maximum Memory:  32K
Logical Memory:  32K
EAE:             Available

Handler Information:

Name   Block  Length  Entry  Block Size
SYS    0123    01     000    00200
NUL    0123    01     034    00200
DTA    0124    01     011    00200
RXA    0125    03     000    00200
RKA    0130    04     000    00200
RKB    0130    04     005    00200
LTA    0134    01     000    00200
DSU    0135    01     075    00200
DSL    0135    01     026    00200
LIN    0136    01     000    00200
LTD    0137    01     000    00400
LID    0140    01     000    00400
FLP    0141    01     000    00200
<FREE> 0142    21     000    00200

Free Slots:      02
```

Figure 2. P?S/8 System Status and Available Resources

Further information is provided on using these operating systems in section 1.61 below.

Note: While it is possible for certain PDP-8 programs to run from nearly all available operating systems by strict adherence to the prevailing memory conventions, each operating system supports specific unique features that require complete program conformance for proper operation. This will be discussed in greater detail in section 1.61 below.

1.4 Using the SIMH simulator for program development

While it is possible to execute existing PDP-8 programs within the SIMH environment, the full implementation (including the PEPS extensions) allows for meaningful development of any PDP-8 program; in many cases, work flow in the PEPS environment exceeds that of the original hardware. Various operating modes are described below.

1.41 Using the SIMH simulator for basic PDP-8 programming

The SIMH PDP-8 simulator has been developed over a course of many years, adding in numerous features and functionality beyond the early implementations. These earlier releases mimicked the most primitive PDP-8 systems which have a limited capability generally referred to (euphemistically) as "the paper-tape operating system". In actuality, this is merely a series of inter-related binary files stored on BIN^[5] format binary paper-tapes intended to be loaded into memory from either the high-speed reader or the low-speed reader of either the Teletype Model 33 ASR or Model 35 ASR. Software commands were then added to the simulator as a stand-in for a hardware front panel; at that point, the scope of what could be accomplished was limited to what could be done with the paper-tape operating system files and any additional paper-tapes created by the user.

While this capability still remains within the SIMH simulator, the addition of a large array of storage and other peripherals allows meaningful usage of more complex systems. Software operating systems that are booted up from the supported devices allows any level of user to accomplish much more than just a basic system (whether simulated or not).

For those who only want to use this limited capability, two alternatives are possible: a) A somewhat tedious process involving constantly changing the SIMH configuration files at each step, b) Using a few commands from the supported operating systems. Minimal knowledge of the supported operating systems allows use of the basic software without the normally associated tedium; if the simulated environment merely consists of a CPU with memory and just a console terminal (generally a Teletype Model 33 ASR) and perhaps a PC04 high-speed reader and punch, this predictably leads to user frustration as is also the case on a real-world equivalent hardware system.

Both the P?S/8 and OS/8 operating systems contain commands to support the rudiments of this primitive software environment. Appendix F details the specific programs used to support paper-tape operations available in both storage-based operating systems; only a small number of easily-learned commands are required to fully support the paper-tape environment.

While the specific programs used in the paper-tape environment are not provided with the PEPS package, they are available from various Internet-based PDP-8 archives. Due to the nature of the overall PEPS environment, these programs may be added to allow easy deployment as required; some additional knowledge of the storage-based operating systems may be required.

1.42 Using the full PEPS environment for PDP-8 programming

As compared to the paper-tape environment, the two provided operating systems (OS/8 and P?S/8) can be called up to perform far more sophisticated operations including running additional programming languages, file editors and other utilities unique to each operating system. Each system is capable of self-contained independent operation including program development. Commands within the PEPS Command directory are available to initiate these operations, as well as transmit files from the host Windows environment to the SIMH environment or vice-versa.

The two operating systems can also exchange the contents of files using available conversion programs, most notably the P?S/8 OS8CON utility for text files.

Binary files can be transferred between the two operating systems using various methods including binary paper-tape punching in one system and then binary file reading in the other. The Command directory includes a shortcut/icon to swap the *PTRIN* and *PTPOUT* files for this specific purpose. Note: Future releases of the P?S/8 operating system will include an additional utility to be known as *BINCON* patterned after OS8CON to more directly effect these transfers. This will allow OS/8 users the ability to more readily utilize the binary output of the superior P?S/8 *PAL* program which includes features not available within OS/8's *PAL8* utility including additional language features, advanced support of the LINC-8 and PDP-12 LINC processor and corrected support of the DTORG directive to allow meaningful usage within both operating systems.

Configurations of the two operating systems are each capable of booting up the other from within the same SIMH session. Appendix G documents the available methods to transfer files between operating systems (some of which are currently under development as of this writing). By using the best features of both systems, the PEPS package can accommodate any level of usage and development realizable in a PDP-8-based environment.

1.43 Host editing and development of PDP-8 source files

while the prevailing operating systems are capable of independent source file development, a distinctive feature of the PEPS environment is the support of host-level file editing with rapid transmission of updated files into the SIMH environment; multiple rounds of editing sessions and program assembly or execution [in the case of high-level languages such as FOCAL] can quickly be achieved. While the standard windows Notepad is the default editing program for the supplied editing commands, any properly qualifying alternative editing program can also be used. Appendix D contains information on some of these editing programs and certain tradeoffs involved when using them.

In general, for any individual file-based programming project, two key icon-based commands are required that are particular to that file. As an alternative, commands are available to quickly setup any one file using generic commands.

Note: While the process of producing a pair of file-specific custom icon-based commands can be adapted from any of the provided examples, it is recommended only advanced users attempt to create their own commands; the process requires significant knowledge of arcane features of Microsoft Windows including working with file attributes from the windows environment (and also from a command prompt) and managing environment variables as used in the internal properties of windows links/shortcuts. Appendix H details the process using one of the provided file projects as an example.

Any user of the PEPS can easily work with any number of files, changing focus between active files as necessary. If the generic commands method is used, the only restriction is the use of one file at a time and limiting file storage to a specific directory. This process is documented in section 1.50 below regarding commands related to the current program filename such as *Set Current Program* and *Edit Current Program*.

As an example of a particular file-based programming project, the *Inactive Programs* directory (accessible from the main Command directory) contains a pair of commands that can be moved to the main Command directory if desired; this is accomplished using standard windows Explorer techniques such as *Cut* or *Copy* and *Paste*:

- 1) Edit PROGRAM.PAL. This command brings up the source file *PROGRAM.PAL* for editing in the standard windows Notepad program. Editing changes can be made at any time. Either saving the file within the editor or exiting the editor is necessary to proceed in the development process of this file.

Note: Appendix C contains information regarding alternate editing programs which can be substituted for windows Notepad on an individualized basis, as well as potential feature tradeoffs that may result. These considerations apply to all file editing situations throughout the PEPS environment. It is strongly recommended only advanced users modify the editing program parameters to call up any particular file editing situation due to potential interaction with open files and other running tasks (such as the SIMH simulator). In certain instances, the host windows version (32-bit or 64-bit) may also be a factor.

If the editor is closed, the contents of the PROGRAM.PAL file is copied to the *PTRIN.TXT* file which, in turn, is used as the dedicated input to the simulated paper-tape reader device (PTR:) within the SIMH simulator environment. Once this is accomplished any of the supported operating systems can be brought up; utilities within each operating system can then process the simulated paper-tape input followed by any desired additional utilities (such as calling up an assembler program).

- 2) Send PROGRAM.PAL. This command copies the contents of the PROGRAM.PAL file to the PTRIN.TXT file as described in 1) above, then a designated operating system is brought up. Any supported operating system can be specified for this operation; however, the same operating system is always used when executing this command.

Because the PROGRAM.PAL file is always copied to the PTRIN.TXT file when using the *Send PROGRAM.PAL* command, it is not necessary to close the editing session, which can stay active in a separate window. By using this method, it is possible to achieve rapid program file development; the editing session need only be closed when done with all development operations.

Note: To use multiple operating systems to process the same file, it is necessary to close the editing session as described in 1) above. At that point, opening a new SIMH session to any operating system [which does not specify any editing or other modification of the PTRIN.TXT file] will pass the last edited file contents in the PTRIN.TXT file. This can be accomplished repeatedly since, in general, only editing sessions upon closing or send command execution will change the PTRIN.TXT file. This can be useful when comparing operating system environments such as OS/8 and P?S/8 when passed the same file or taking advantage of features unique to each operating system.

The underlying implementation of these two commands are two related batch files called *Edit and Copy PROGRAM.PAL* and *Send PROGRAM.PAL*. These files (and the shortcuts/links to them generally placed in the Command directory when actively used) need to be adapted to create an equivalent set of batch files and shortcuts/links for each user-defined file; the provided files are only meant to provide a working example.

The user can remove all such unused components as necessary to avoid clutter when adding in new user-defined components. Note: It is recommended that unneeded shortcuts/links be moved to the Inactive Programs directory already provided for this purpose; the underlying batch files require very little storage and might be useful for future user file development projects.

Note: The PROGRAM.PAL file, as provided in the PEPS package, is a dummy file intended for use by those who want to quickly develop a short program in *PDP-8 Assembly Language* (PAL). By foregoing a distinctive name, any user can quickly develop this file as far as deemed necessary. As required, the file can later be copied to another directory (and renamed as the user wishes) using Windows Explorer techniques.

Depending on requirements, future work can be setup in a variety of ways. For slightly larger projects, it is recommended that the user deploy the current program filename technique as described above. While this requires some slight additional setup effort on the part of the user, this is easily mastered by beginners after some minimal experience with the PEPS environment.

Advanced users can create custom command pairs patterned after the method used to develop PROGRAM.PAL while applying their own unique filenames. Using additional advanced techniques, files can be located somewhere within the recommended file structure group (*PDP-8\Source Files*) or on other windows directories/devices as deemed appropriate.

Appendix H documents the relevant parameters that must be set to implement any of these methods; not every project should necessarily be setup using the same overall method. Short-term projects using PROGRAM.PAL can be used instantly while named files residing in the designated current program files directory may describe a multiple file project. Only the most advanced projects may require resorting to the custom-named files method.

Each method has a slightly different set of tradeoffs; as user experience grows, it will become apparent which approach serves best for any programming project. In certain cases, working with source files created by others would contemplate little to no file editing once the proper file contents are confirmed. The custom command pairs method would generally be too tedious in this situation; copying all such files to the current program files directory would likely suffice.

1.50 Operating the PEPS from the Command directory

The Command directory arrangement shown in Figure 3 includes all commands associated with the current release. This includes working commands provided as examples illustrating host editing of specified text files and the transmission of updated files into the SIMH environment using the supplied operating systems. Additional commands are provided to process various files returned from the SIMH environment as well as perform other functions as described below.

The specific details of the Command window are subject to user preference considerations with regard to approaching the Windows desktop. The commands shown in Figure 3 are configured according to certain windows parameters and other factors currently in effect:

- a) Medium icons are in use; other available alternatives include smaller and larger icons, full details and a larger tiled format; some of these alternatives will fully display the captions labeling all icons regardless of label character count (albeit likely occupying a larger portion of the overall windows desktop).

In this example, one of the icons was deliberately selected (thus the displayed color is darker) for the purpose of expanding the longer caption text. Due to the way windows operates, when not selected, the caption text may become truncated unless one of the other view alternatives is used.

In general, most of the captions are completely shown even when not selected; the medium icons and supplied captions represent a reasonable compromise of readability and clutter avoidance.

- b) All command icons are positioned by asserting *Align to Grid* with the associated *Auto Arrange* option disabled. Settings to accomplish this are available within the window view particulars available in all supported windows versions [perhaps requiring a small patch to the desktop]. Note: Third-party patches for the affected systems are described in Appendix I.
- c) The grouping of icons in the Command window is primarily by similarity of function. As necessary, empty positions are deliberately reserved for the purpose of future expansion. Additional commands will likely be added in future releases. Several examples of specific file editing and transmitting the updated file to the SIMH simulator are provided. Additionally, a series of commands is provided to implement generic file processing requiring no changes to the Command window.

- d) The general icon color scheme chosen includes specific colors for related functions as well as colors associated with two of the supported operating systems [OS/8 and P?S/8 configured without limitations]. These color cues assist the user in selecting the desired function. Advanced users can modify any of these considerations; however, beginners are advised to work with the standard release as documented here.



Figure 3. PEPS Command directory window

Advanced users are encouraged to create custom icons for new command shortcuts/links added to the Command directory; new icon files must be placed in the existing icon directory (*Links and Support Files*) prior to editing shortcut properties, generally using a copy of an existing shortcut/link as a template. This will greatly simplify the editing process; otherwise, windows acts in a presumptive manner which can be quite frustrating. Note: This is a severe limitation of Windows that cannot easily be overcome. By using the provided directory for all icon files, the path strings within the shortcut can be safely edited changing only the trailing characters in the path string (which is the icon filename) while maintaining the identical leading portion of the path string. More information on creating custom shortcuts/links is available in Appendix H.

1.60 Description of individual commands within the Command window

Note: The five rows of the command window shown in Figure 3 are described below in left to right order.

1.61 Supported Operating Systems

Note: As shown in Figure 3, commands to launch various operating systems are located in the top row of the Command directory.

- a) OS/8. This is generally OS/8 Version 3D (also known as OS/8 Version 3Q). In specific instances, system programs have been included from newer OS/8 family releases known to correct bugs in the original versions. Certain non-DEC programs are also provided to perform additional tasks, such as the Kermit-12 utilities written by Charles Lasner. These programs are needed to process specific files obtained externally from other users usually by means of the Internet.

The included configuration of OS/8 boots to the RK8E/RK05, generally mounted on RK05 unit 0. By changing the configuration of mounted drives, it is possible to boot OS/8 on any RK05 drive 0-3.

Due to the manner in which the host windows environment passes files to the SIMH simulator, it is necessary to include the standard OS/8 PTR: handler to process paper-tape input.

Note: Due to a quirk of the PTR: handler, the very last character of the simulated PTR: character stream (contents of the *PTRIN* file in the *SIMH_Working_Files* directory) will be ignored. As such, any file obtained from the PTR: handler will be truncated. If the passed file is an ASCII text file, the lost character will generally be a <LF> character, which can be restored by passing the file through OS/8 PIP or various editing programs such as OS/8 TECO. Certain programs which process <CR> characters and ignore <LF> characters will function correctly when processing these files despite the slight defect. Binary files with generous leader/trailer characters will not be affected by the PTR: quirk.

- b) P?S/8. This is Version 8Z of P?S/8 configured for the TC01/TC08 DECTape with up to eight logical drives [0-1]. Over the years, various patches have been applied to Version 8Z to correct certain known bugs. An ECO log file is available to detail what was done since the original release of Version 8Z on 23-Feb-1988. Note: Future support of P?S/8 within the PEPS environment will be upgraded with the release of P?S/8 Version 9A which is currently under development.

Note: Both OS/8 and P?S/8 are capable of booting any available operating systems directly using simple commands or running short boot-up programs which are provided in each system. In the case of the RK8E/RK05 cartridge disk device, this will allow boot-up to any of the four possible drives if a suitable cartridge file is mounted accordingly.

- c) P?S/8 with the line printer disabled. In earlier releases of P?S/8, many programs automatically directed output to the device 66 line printer if it was available and online. To prevent this action, it was necessary to disable the line printer either by powering down or placing the device in an offline state.

As of the release of P?S/8 Version 8Z, most of these problems have been eliminated by implementing command-line switches in the affected programs that can force output to the console printer if desired. To allow testing of any remaining problems, P?S/8 may be started with the device 66 line printer logically disabled in the associated SIMH configuration file. While fully supported, this system is of little interest to most PEPS users; once all known issues are resolved, this mode of operation will likely be withdrawn from future releases of the PEPS package or moved to the Inactive Commands directory.

Note: Future releases of the PEPS package will include additional operating systems such as P?S/8 configured for the TD8E/TU56, RK8E/RK05 or RX8E/RX01. Additional OS/8 systems can be added that can boot to devices other than the RK8E/RK05 such as the RX8E/RX01 [or support different non-system handlers]. As the Command directory window becomes more complex, it may become necessary to include an internal folder for choice of operating system to avoid clutter; advanced users may also require internal folders containing additional file development icons/links (other than a few currently present in the main folder) for similar reasons. With experience, all users can determine which windows desktop particulars serve their needs in an efficient manner; consider the provided Command directory (as shown in figure 3) as merely a starting point which can evolve as the user's needs change.

1.62 Generic file handling and related commands

Note: As shown in Figure 3, all generic file handling and related commands are located in the second row of the Command directory.

The PEPS environment includes an internal folder for files currently under development which is referenced as the *Current_Program_Files* directory. This is in sharp contrast with files stored in a more general collection of external directories organized by user preference; a partial implementation of these directories is provided with each release of the PEPS including certain example files that are accessible with explicit commands. This file structure should serve as a guide to additional user directory organization and file development.

In the course of accessing these files, it will likely become necessary to interact with both the external directories and the *Current_Program_Files* directory. Commands in this section are available to work with individual files in the *Current_Program_Files* directory as well as interact with the external directories using Windows Explorer techniques.

- a) Edit Current Program. The current named file is opened in a designated editor, in this case the standard windows Notepad program. The Current Program filename is set using the *Set Current Program* command described in c) below.

Closing the editing program generally saves the updated state of the current named file; the file is then copied to the *PTRIN.TXT* file used as the input to the PTR: device within the SIMH simulator environment. This allows any operating system to be launched with the contents of the current named file already set up. However, it is not necessary to close the editing program; if the file is saved using commands within the editor, the *Send Current Program* command described in b) below can be used to allow rapid editing operations on the file after the designated operating system has processed the file as needed. This is the fastest way to develop most PDP-8 programs while also taking advantage of editing the file in the host Windows environment.

- b) Send Current Program. The contents of the current named file are copied to the *PTRIN.TXT* file used as the input to the PTR: device within the SIMH environment. The SIMH simulator is then launched in a separate window, booting to the designated operating system as defined in the corresponding underlying batch file associated with this command [*Send Current Program.BAT*]. The Current Program filename is set using the *Set Current Program* command described in c) below.

Note: As released, P?S/8 with standard LPT: support is the designated operating system for use with this command. By applying a small change to the *Send Current Program.BAT* file, any available operating system can be chosen. Several operating system choices are already provided within the batch file; all inactive choices should be preceded with a REM statement to simplify further changes in the future.

- c) Set Current Program. The Current Program filename [used in a) and b) above] is set by invoking this command.

Note: The Current Program filename is permanently stored within the PEPS package. This command is the only method available to update the filename, which is internally stored within a key parameter file of the package; this applies even if the entire PEPS environment is transferred to a different Windows system.

As shown in Figure 4 below, the default Current Program file name is *HELLOWORLD.PAL*. This command allows the filename to be changed to any desired combination of alphanumeric characters in upper or lower case; while embedded space characters are not allowed, several additional characters (such as [and]) are permitted.

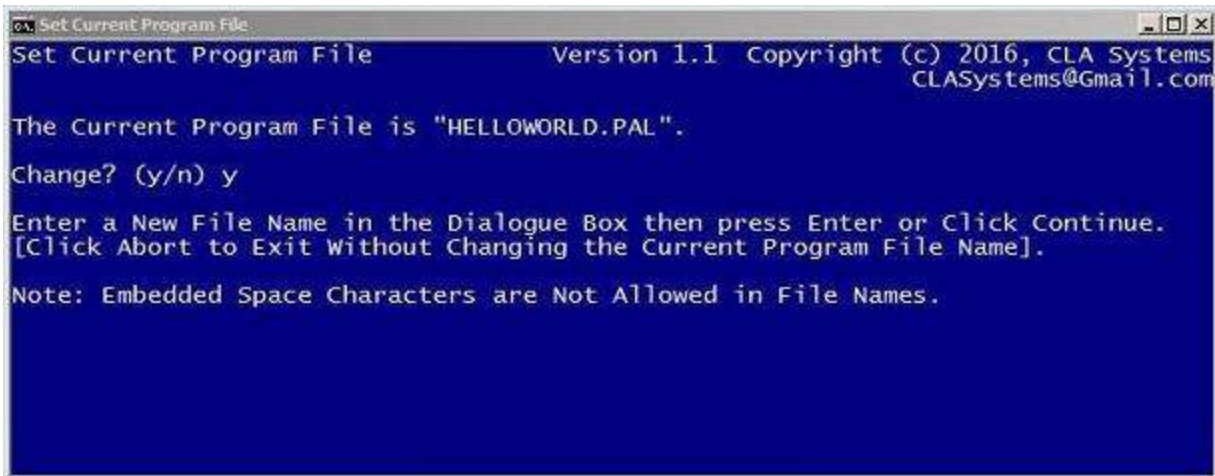


Figure 4. Set Current Program File Screen

As shown in Figure 5 below, the Current File name is being changed to *MYPROGRAM.PAL* by manual entry into the associated dialogue box launched by the command when a change is requested. The user can abort the process by clicking on the *Abort* button [or the "X" in the upper right corner] or confirm the requested new file change by clicking on the *Continue* button [or pressing the *Enter* key on the keyboard].

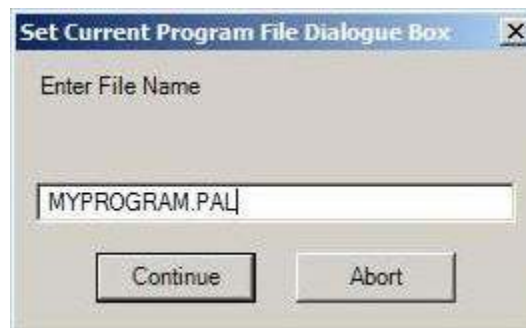


Figure 5. Set Current Program File Dialogue Box

Once the filename request is processed, the new value of the Current Program filename applies. As shown in Figure 6 below, if the new Current Program file does not exist, a new file will automatically be created in the *Current_Program_Files* directory. When the command exits, the Current Program filename will permanently be changed to *MYPROGRAM.PAL* unless and until another *Set Current Program* command is executed.

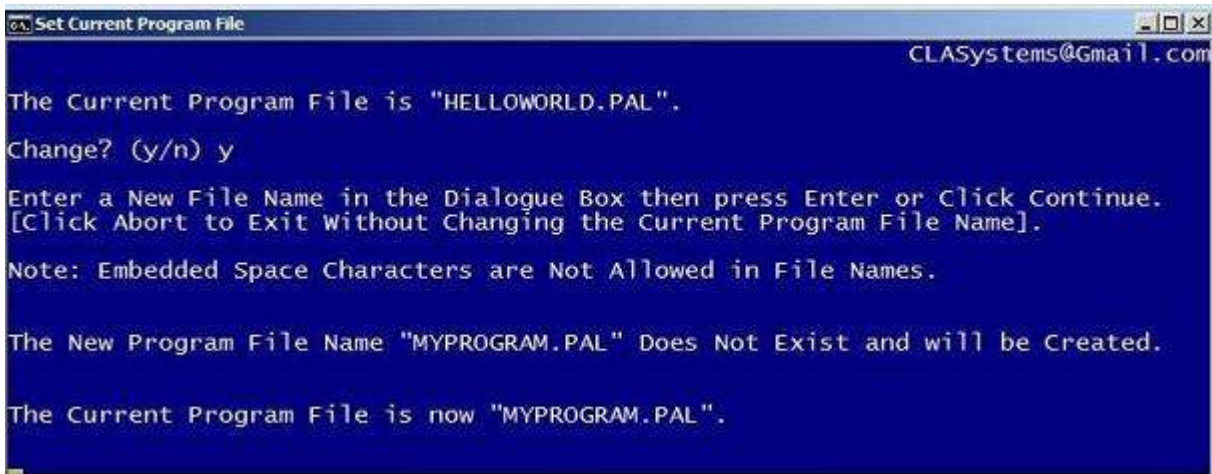


Figure 6. Set Current Program File Screen after filename change

- d) Explore Current Program Files. The Current Program Files directory [*Current_Program_Files*] is open in a separate window. Files can be transferred in or out of this directory from other directory folders as required using standard windows Explorer techniques.

Note: this command is often used in conjunction with the *Explore Additional Program Files* command as described in f) below.

- e) List Current Program Files. As shown in Figure 7 below, a listing file of the *Current_Program_Files* directory is created then opened in a separate window using the standard windows Notepad utility. This file may be modified as necessary and saved or printed if desired.

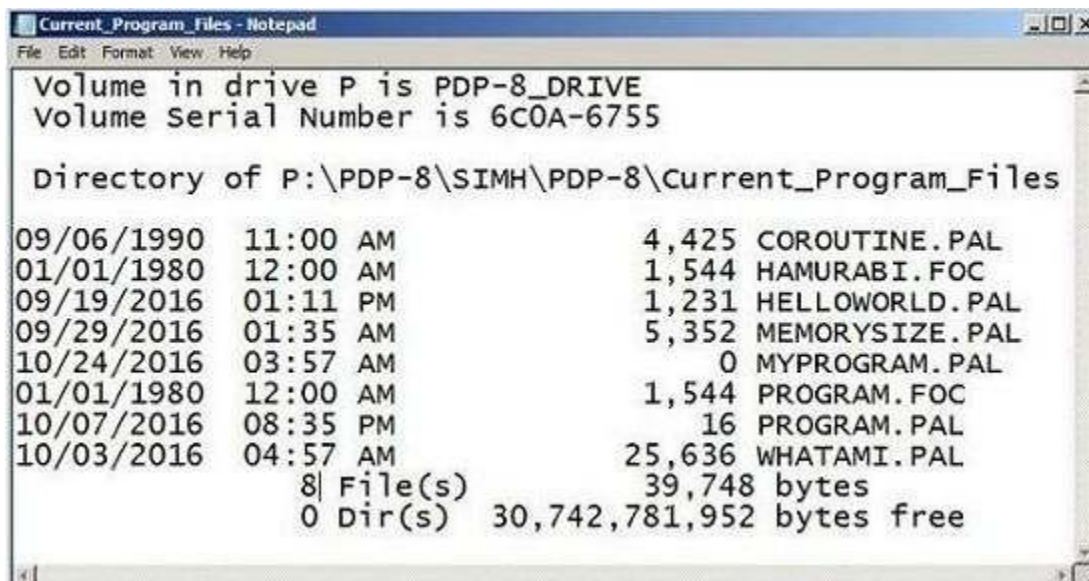


Figure 7. Current Program Files Directory Listing

Note: The *Set Current Program* dialogue box as described in c) above can be filled by applying copy and paste techniques to this window to aid in minimizing manual entry where applicable. Existing filename characters can be used to [partially] set the Current Program filename without having to manually type the entire filename. For example, the existing file name *PROGRAM.PAL* can be copied and then modified into *PROGRAM1.PAL* or *PROGRAM2.PAL*, etc.

- f) Explore Additional Program Files. A separate windows Explorer window is opened to the *PDP-8\Source Files* directory. It is recommended that all files oriented to PDP-8 projects be organized into appropriate sub-folders within this directory. Files stored in this area can be interchanged with those of the *Current_Program_Files* directory using windows Explorer techniques.

1.63 Custom commands for individual file development

Note: As shown in Figure 3, all custom commands for individual file development are located in the third row of the Command directory window.

Some of the command examples supplied in this section are associated with the OS/8 operating system while others are associated with the P?S/8 operating system. While this generally pertains to which operating system is chosen when sending the file via the *PTRIN.TXT* mechanism, the corresponding command icons/links are color-coded for consistency. Black icons are selected for files to be developed using OS/8 while blue icons are selected for files to be developed using P?S/8.

Advanced users can derive commands for development of additional files using any color scheme [or alternate icon set] deemed appropriate. Appendix E provides information for advanced users to aid in creating new batch files and corresponding command shortcut/icon entries. The provided files described in this section can be used as templates for any custom additions to the user's system.

- a) Edit *COROUTINE.PAL*. The text file *\PDP-8\Source Files\Generic Source Files\COROUTINE.PAL* is opened in a new window using windows Notepad [the designated text editing program]. All particulars of this command are defined within the *Edit and Copy COROUTINE.PAL.BAT* file in the *SIMH Batch Files* directory. As explained above with regard to the *Edit Current Program* command, Appendix C contains information regarding alternative editing programs.

Closing the editing program generally saves changes made to the *COROUTINE.PAL* file; the file is then copied to the *PTRIN.TXT* file used for input to the PTR: device within the next SIMH simulator session. This allows any operating system to be launched with the contents of *COROUTINE.PAL* already set up for file capture within the selected PDP-8 operating system.

For most editing situations it is not necessary to close the editing program each time changes are made. By merely saving the file, the *Send COROUTINE.PAL* command described in b) below can be used to allow rapidly repeated editing operations on the file under development after the designated operating system has processed the file leading to the need for additional editing. This is the fastest way to develop most PDP-8 programs while also taking advantage of editing the file in the host windows environment.

- b) *Send COROUTINE.PAL*. The contents of the *\PDP-8\Source Files\Generic Source Files\COROUTINE.PAL* file are copied to the *PTRIN.TXT* file used for input to the PTR: device within the SIMH environment. The SIMH simulator is then launched in a separate window, booting to the designated operating system as defined in the corresponding underlying batch file associated with this command [*Send COROUTINE.PAL.BAT*]. Note: while OS/8 is the default operating system for this function, a one line edit to the batch file can be made to boot to P?S/8 or other operating system; this option exists in all analogous batch files to provide free choice of booted operating system when sending files via the PTR: mechanism.

Note: Since a) and b) above refer to a file developed within the OS/8 environment, the associated icons for these commands are black color.

- c) *Edit MEMORYSIZE.PAL*. The text file *\PDP-8\Source Files\Generic\Source Files\MEMORYSIZE.PAL* is opened in a new window using windows Notepad [the designated text editing program]. All particulars of this command are defined within the *Edit and Copy MEMORYSIZE.PAL.BAT* file in the *SIMH Batch Files* directory. As explained above with regard to the *Edit Current Program* command, Appendix C contains information regarding alternative editing programs.

Closing the editing program generally saves changes made to the *MEMORYSIZE.PAL* file; the file is then copied to the *PTRIN.TXT* file used for input to the PTR: device within the next SIMH simulator session. This allows any operating system to be launched with the contents of the *MEMORYSIZE.PAL* already set up for file capture within the designated PDP-8 operating system.

For most editing situations it is not necessary to close the editing program each time changes are made. By merely saving the file, the *Send MEMORYSIZE.PAL* command described in d) below can be used to allow rapidly repeated editing operations on the file after the designated operating system has processed the file leading to the need for additional editing. This is the fastest way to develop most PDP-8 programs while also taking advantage of editing the file in the host Windows environment.

- d) *Send MEMORYSIZE.PAL*. The contents of the *\PDP-8\Source Files\Generic Source Files\MEMORYSIZE.PAL* file are copied to the *PTRIN.TXT* file used for input to the PTR: device within the SIMH environment. The SIMH simulator is then launched in a separate window, booting to the designated operating system as defined in the corresponding underlying batch file associated with this command [*Send MEMORYSIZE.PAL.BAT*]. Note: while OS/8 is the default operating system for this function, a one line edit to the batch file can be made to boot to P?S/8 or other operating system; this option exists in all analogous batch files to provide free choice of booted operating system when sending files via the PTR: mechanism.

Note: Since c) and d) above refer to a file developed within the OS/8 environment, the associated icons for these commands are black color.

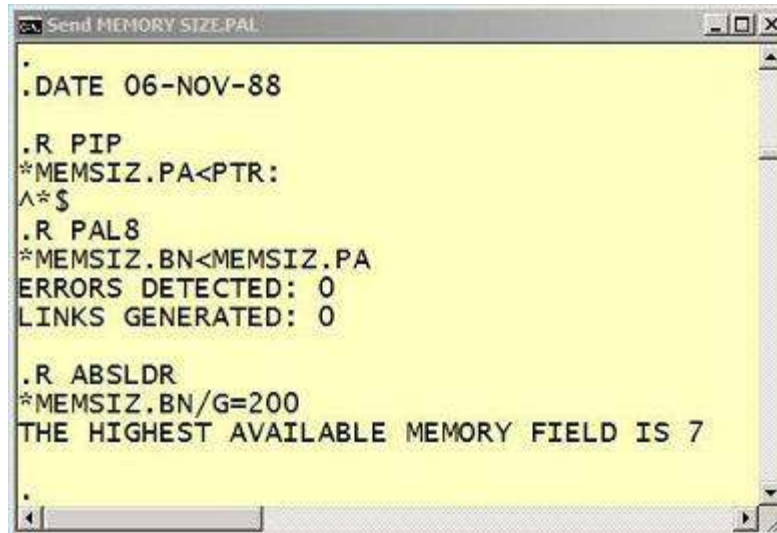
An OS/8 session using *MEMORYSIZE.PAL* passed as the input to the PTR: device is shown in Figure 8. The various steps taken are documented below. Note: The session window is labeled *Send MEMORY SIZE.PAL* because the corresponding icon label contains space characters to improve readability.

- i) The OS/8 date is set to 06-Nov-88. This date was chosen because it allows the day of the week to correspond to that used in the year 2016. OS/8 dates are not maintained across SIMH sessions.

Note: OS/8 only allows dates in the range of 01-Jan-1970 through 31-Dec-2001 with eight-year group anomalies for all dates past 31-Dec-1977.

- ii) The OS/8 *PIP.SV* utility is then executed to create the file *MEMSIZ.PA* from the paper-tape input stream setup when the SIMH session was initiated.
- iii) The OS/8 *PAL8.SV* utility is then executed to assemble the contents of the *MEMSIZ.PA* file into the *MEMSIZ.BN* binary file.
- iv) The OS/8 *ABSLDR.SV* utility is used to load the *MEMSIZ.BN* binary file into memory; by specifying */G=200*, the program is automatically started.

- v) The console output produced by program execution follows. Since the SIMH simulator is configured to provide a 32K memory environment, there are eight memory fields available numbered 0 through 7; the highest memory field is correctly indicated as 7.



```
.DATE 06-NOV-88

.R PIP
*MEMSIZ.PA<PTR:
^*$
.R PAL8
*MEMSIZ.BN<MEMSIZ.PA
ERRORS DETECTED: 0
LINKS GENERATED: 0

.R ABSLDR
*MEMSIZ.BN/G=200
THE HIGHEST AVAILABLE MEMORY FIELD IS 7
```

Figure 8. OS/8 session using MEMORYSIZE.PAL

- e) Edit WHATAMI.PAL. The text file *\PDP-8\Source Files\Generic\Source Files\WHATAMI.PAL* is opened in a new window using windows Notepad [the designated text editing program]. All particulars of this command are defined within the *Edit and Copy WHATAMI.PAL.BAT* file in the *SIMH Batch Files* directory. As explained above with regard to the *Edit Current Program* command, Appendix C contains information regarding alternative editing programs.

Closing the editing program generally saves changes made to the *WHATAMI.PAL* file; the file is then copied to the *PTRIN.TXT* file used for input to the PTR: device within the next SIMH simulator session. This allows any operating system to be launched with the contents of the *WHATAMI.PAL* already set up for file capture within the designated PDP-8 operating system.

For most editing situations it is not necessary to close the editing program each time changes are made. By merely saving the file, the *Send WHATAMI.PAL* command described in f) below can be used to allow rapidly repeating editing operations on the file after the designated operating system has processed the file leading to the need for additional editing. This is the fastest way to develop most PDP-8 programs while also taking advantage of editing the file in the host windows environment.

- f) Send *WHATAMI.PAL*. The contents of the *\PDP-8\Source Files\Generic Source Files\WHATAMI.PAL* file are copied to the *PTRIN.TXT* file used for input to the PTR: device within the SIMH environment. The SIMH simulator is then launched in a separate window, booting to the designated operating system as defined in the corresponding underlying batch file associated with this command [*Send WHATAMI.PAL.BAT*]. Note: While P?S/8 is the default operating system for this function, a one line edit to the batch file can be made to boot to OS/8 or other available operating system; this option exists in all analogous batch files to provide free choice of booted operating system when sending files via the PTR: mechanism.

Note: Since e) and f) above refer to a file developed within the P?S/8 environment, the associated icons for these commands are blue color.

A P?S/8 session using *WHATAMI.PAL* passed as the input to the PTR: device is shown in Figure 9. The various steps taken are documented below:

- i) The P?S/8 date was set to 06-Nov-2016 in a prior SIMH session. Unlike OS/8, P?S/8 retains the current date permanently until changed.

Note: P?S/8 date handling in Version 8Z can be extended through 2099 by applying maintenance patches every eight years. Future releases will extend date handling through 31-Dec-2411 without requiring maintenance.

- ii) The *WHATAMI.PAL* file was processed into the eight *Tiny File System* [TFS] files *WHT[1]*, *WHT[2]*, *WHT[3]*, *WHT[4]*, *WHT[5]*, *WHT[6]*, *WHT[7]* and *WHT[8]* from the paper-tape input stream setup in a prior SIMH session.

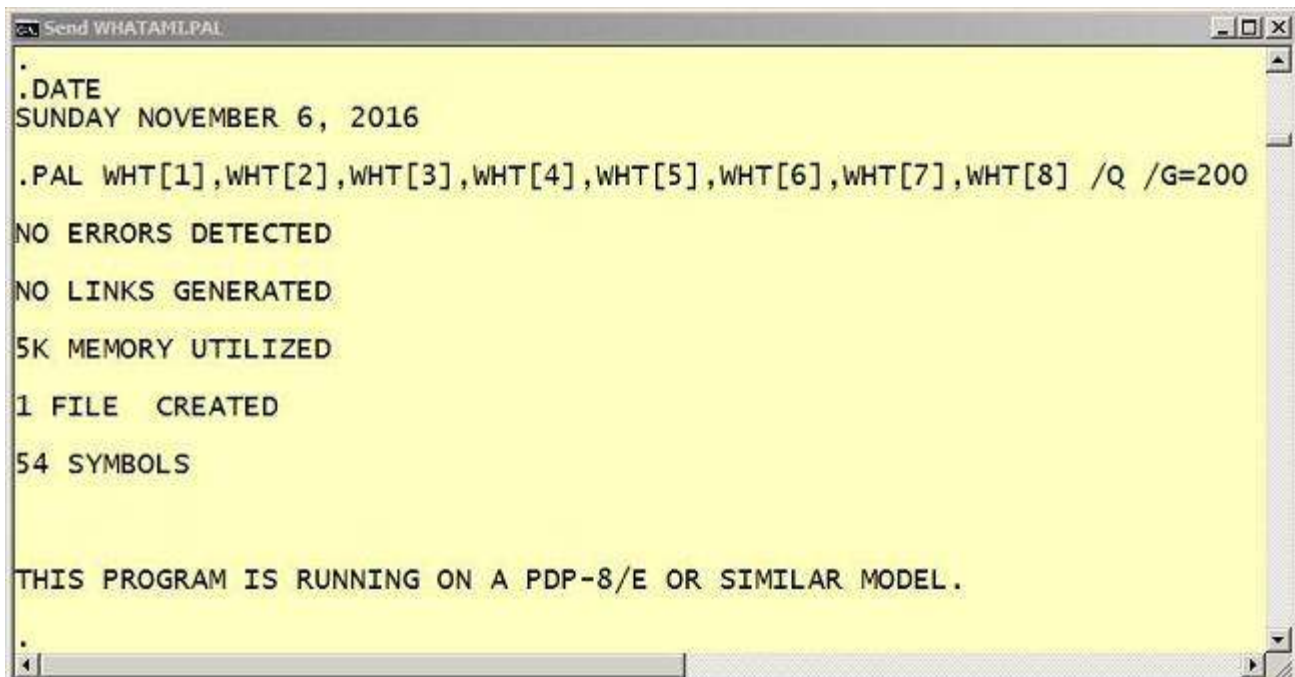
Note: P?S/8 also supports extended-length files. If this method is chosen, this results in a single extended-length file such as *WHTAMI*. Since P?S/8 supports multiple file structures, the user can determine which is the most appropriate for a given situation.

The eight TFS files are passed to the P?S/8 *PAL* assembler program; command-line option switches initiate the assembly process followed by an automatic chain to the P/S/8 *BIN* binary file loading program; the default binary file % is used as the output target of the assembly and then the binary input file passed to *BIN*. Since a starting address is passed using */G=200*, the program starts automatically after loading.

Note: The assembler statistics show there were no assembly errors. Had there been any errors, the automatic chain to *BIN* would have been prevented.

- iii) The WHATAMI.PAL program is designed to detect the specific PDP-8 hardware model the program is running on. Since the SIMH simulator is configured to support the KK8F CPU, the program indicates it is running on a PDP-8/E or similar model.

Note: As explained in section 1.1 above, and as provided in the default configuration, the SIMH simulator provides full compatibility with the KK8F CPU. As such, it is not possible for a software program to differentiate between the PDP-8/E, PDP-8/F, PDP-8/M and the PDP-8/A-620 series models.



```
.DATE
SUNDAY NOVEMBER 6, 2016

.PAL WHT[1],WHT[2],WHT[3],WHT[4],WHT[5],WHT[6],WHT[7],WHT[8] /Q /G=200

NO ERRORS DETECTED
NO LINKS GENERATED
5K MEMORY UTILIZED
1 FILE CREATED
54 SYMBOLS

THIS PROGRAM IS RUNNING ON A PDP-8/E OR SIMILAR MODEL.
```

Figure 9. P?S/8 session using WHATAMI.PAL

- g) Edit FOCAL Program. The text file `\PDP-8\SIMH\PDP-8\Current_Program_Files\PROGRAM.FOC` is opened in a new window using windows Notepad [the designated text editing program]. All particulars of this command are defined within the *Edit and Copy FOCAL Program.BAT* file in the *SIMH Batch Files* directory. As explained above with regard to the *Edit Current Program* command, Appendix C contains information regarding alternative editing programs.

Closing the editing program generally saves changes made to the *PROGRAM.FOC* file; the file is then copied to the *PTRIN.TXT* file used for input to the PTR: device within the next SIMH simulator session. This allows any operating system to be launched with the contents of the *PROGRAM.FOC* already set up for file capture within the designated PDP-8 operating system.

For most editing situations it is not necessary to close the editing program each time changes are made. By merely saving the file, the *Send PROGRAM.FOC* command described in h) below can be used to allow rapidly repeated editing operations on the file after the designated operating system has processed the file leading to the need for additional editing. This is the fastest way to develop most PDP-8 [and FOCAL] programs while also taking advantage of editing the file in the host windows environment.

- h) Send FOCAL Program. The contents of the `\PDP-8\SIMH\PDP-8\Current_Program_Files\PROGRAM.FOC` file are copied to the *PTRIN.TXT* file used for input to the PTR: device within the SIMH environment. The SIMH simulator is then launched in a separate window, booting to the designated operating system as defined in the corresponding underlying batch file associated with this command [*Send FOCAL Program.BAT*]. Note: while P?S/8 is the default operating system for this function, a one line edit to the batch file can be made to boot to OS/8 or other operating system; this option exists in all analogous batch files to provide free choice of booted operating system when sending files via the PTR: mechanism.

Note: Since g) and h) above refer to a file developed within the P?S/8 environment, the associated icons for these commands are blue color. However, since these commands are oriented towards FOCAL programs, a somewhat lighter shade of blue was chosen.

The default contents of the *PROGRAM.FOC* file is an accurate copy of the historic DEC *King of Sumeria* game known as *HAMURABI*. This file is also included in the `\PDP-8\Source Files\PQS8 FOCAL - Related Files` directory as *HAMURABI.FOC*.

A P?S/8 session using *PROGRAM.FOC* passed as the input to the PTR: device is shown in Figure 10. The various steps taken are documented below:

- i) As described above, the paper-tape input stream setup in a prior SIMH session is the contents of *PROGRAM.FOC* which in turn is a copy of *HAMURABI.FOC*. As is usually the case with most short FOCAL programs, the entire program was converted into a single TFS file which was named *HMRABI*.
- ii) The *HMRABI* file is then passed to P?S/8 *FOCAL* which is a binary compatible superset of the original DEC FOCAL, 1969 release. The contents of passed binary and text files are used to implement custom binary patches to FOCAL and pass FOCAL programs respectively. In this instance, no binary patches are required.

Various command-line switches, both present and deliberately omitted, control the precise way FOCAL operates. In this instance, the Sine and Arctangent option groups are not loaded to maximize the memory space allocated to the program, variables and pushdown stack.

The source file contents do not echo on the console device during the loading process [this can be enabled by changing the command-line options]. Automatic program startup after displaying the FOCAL prompt character is also initiated by the selected command-line options.

- iii) The running program presents various options to the town manager ["HAMURABI"] requiring some thought on how best to proceed allocating the town's resources. Execution is stopped by the use of the Control-C character to abort the FOCAL program.
- iv) Users quite familiar with the HAMURABI game as documented in various DEC handbooks may notice small differences in the output as shown in Figure 10. The original FOCAL, 1969 supplies a leading = character on all numerical output; the output format includes an extra leading space which tends to make the values slightly harder to read.

It is the consensus of the FOCAL user community that these features are quirks which should be suppressed; as such, P?S/8 FOCAL removes them by default. Command-line options are available to restore either or both of these features if desired.

```
. FOCAL HMRABI /F /G
*
HAMURABI:
LAST YEAR
    0 STARVED,
    5 ARRIVED,
POPULATION IS 100
THE CITY OWNS 1000 ACRES.
WE HARVESTED 3 BUSHEL PER ACRE;
RATS ATE 200 BUSHEL, YOU NOW HAVE
2800 BUSHEL IN STORE.
HAMURABI: LAND IS TRADING AT 21 BUSHEL PER ACRE;
HOW MANY ACRES OF LAND DO YOU WISH TO BUY?
: ^C
```

Figure 10. PDP-8 FOCAL session using PROGRAM.FOC

1.64 special-purpose utility commands

Note: As shown in Figure 3, all special-purpose utility commands are located in the fourth row of the Command directory window.

All of the commands in this section are permanent components of the PEPS package concerned with the files used to intercommunicate between the host windows environment and the SIMH simulated environment; these files are located in the `\PDP-8\SIMH_Working_Files` directory. The commands are used to carry out various functions on these files needed when using any of the supported operating systems. The specific files are described below:

- 1) LPTOUT.TXT - This file is used to capture the output directed to the device 66 line-printer. Various commands are implemented to manage the contents of this file.

- 2) *PTPOUT.TXT* - This file is used to capture the output directed to the high-speed punch [PTP:] device. While the file extension used might imply printable text contents, the actual file contents could be non-printable binary coding depending on specific circumstances. Actual text contents can be opened in any designated editing program capable of processing ASCII text characters in the file. Windows Notepad is the default program used throughout the PEPS package for such editing purposes. On an case by case basis, advanced users can modify the underlying command files to use a qualified alternate text editing program if desired.
- 3) *PTRIN.TXT* - This file is used to transmit input data to the high-speed reader [PTR:] device. As described in 2) above, the file extension might imply printable text contents. However, the actual file contents could be binary data such as a standard PDP-8 binary file in *BIN*^[4] format. Generally the user determines the specific contents of the *PTRIN.TXT* file and the set of utilities provided within the designated operating system that will process the file as intended.

The various commands described in this section manage nearly all situations related to these key files. In certain rare instances, it may become necessary to use one of the miscellaneous commands described below in section 1.65 to accomplish certain tasks using Windows Explorer. This would generally concern transfer of files obtained via the Internet or other external resources. Appendix Q contains more information on optional commands to be included in this section or currently under development. If specific utilities are proposed and implemented, command links can be added to this section in future releases.

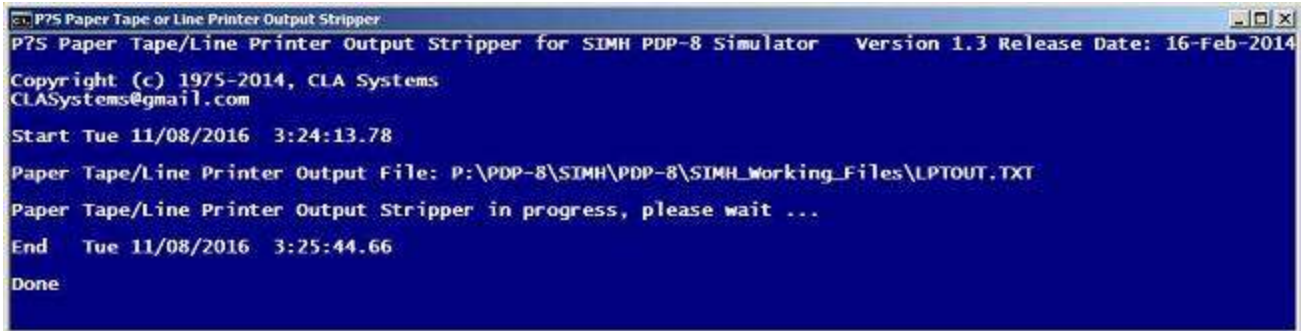
- a) Clear LPT Output. The *LPTOUT.TXT* file is emptied. This command is needed to ensure there are no residual characters in the file resulting from previous SIMH sessions. If the purpose of the next SIMH session is to produce a listing file directed to the device 66 LPT: device, this command must precede launching the SIMH simulator.
- b) Strip LPT Output. The *LPTOUT.TXT* file often contains known artifacts created by imperfect operation of various output utilities in the supported operating systems. Left as created by the issuing utility, the file is often useless in the Windows environment.

Some of the problems with undesired *LPTOUT.TXT* file contents arise from necessary implementation restrictions within the original real-world PDP-8 environment; certain utilities time out the LPT: device and then fall back on the console printer should the LPT: not be available. In some instances, these characters would have been ignored in the sense that no artifacts would appear on the printed page of a real-world system line printer; however, the SIMH simulator captures all characters sent to the LPT:. As such, it is generally necessary to remove these unwanted characters from the *LPTOUT.TXT* file. In certain situations, the artifact characters, while present, are truly unfathomable or only work with specific printer models yet were never discovered during the economic lifetime of these operating systems. In the case of certain P?S/8 line printer quirks, the original development environment often lacked access to a line printer and the program implemented the best guesses of the programmer. Some of these issues resulted in wasting a sheet of line printer paper; the intended output started on the next page at the top of form point.

To solve all of the problems discussed above [and arcane additional issues not documented here], the PEPS package includes the *STRIP.TEC* macro designed to work with *TECOC*, a freeware implementation of DEC standard *TECO* for Windows. Once a file is processed by *STRIP.TEC*, all such undesired output is eliminated; the resultant file is now ready for further processing such as input to a word-processor for the purpose of printing the file in windows. In some cases, post-processing might include conversion to PDF format for distribution to a wider audience or file archiving.

Running the *STRIP.TEC* macro on typical text files is a time-consuming process; the actual execution time depends on the Windows system processing power, *LPTOUT.TXT* file length and other considerations. As shown in Figure 11, a separate window is created to display the stripping process; the starting and ending times are displayed during the program execution to aid the user in determining future processing time should the file [or a subsequent version after a source file edit] require additional stripping.

While the example shown in Figure 11 took just over 90 seconds to complete, it is not uncommon for very long assembly listings to take over five minutes for conversion into a properly formatted file on typical Windows systems.



```
P7S Paper Tape or Line Printer Output Stripper
P7S Paper Tape/Line Printer Output Stripper for SIMH PDP-8 Simulator  Version 1.3 Release Date: 16-Feb-2014
Copyright (c) 1975-2014, CLA Systems
CLASystems@gmail.com
Start Tue 11/08/2016 3:24:13.78
Paper Tape/Line Printer Output File: P:\PDP-8\SIMH\PDP-8\SIMH_Working_Files\LPTOUT.TXT
Paper Tape/Line Printer Output Stripper in progress, please wait ...
End Tue 11/08/2016 3:25:44.66
Done
```

Figure 11. LPTOUT.TXT file stripping session

- c) Open LPT Output. The *LPTOUT.TXT* file is opened in the standard Windows Notepad editor. This allows immediate display of the consequences of processing such as assembling the source file currently under development.

Note: while there is generally no need to modify the *LPTOUT.TXT* file, editing operations are freely allowed; the file may be printed if desired. In many circumstances, the font size must be reduced to obtain a complete display on the windows desktop without resorting to horizontal scrolling. This typically occurs because assembly language utilities add additional content at the left margin of every source file line.

while alternative editors may be useful for typical editing situations, most other qualifying editing utilities may either cause all open text files to share a common window [by selecting one of a set of multiple tabs] or not support multiple font sizes in a single session. Unless the alternate program can support these additional features, it is recommended that windows Notepad continue to be used for this purpose.

- d) Clear PTP Output. The *PTPOUT.TXT* file is emptied. This command is needed to ensure there are no residual characters in the file resulting from previous SIMH sessions. If the purpose of the next SIMH session is to produce a text file directed to the PTP: device, this command must precede launching the SIMH simulator.
- e) Strip PTP Output. The *PTPOUT.TXT* file is stripped of unwanted artifacts; other than the change of target file from *LPTOUT.TXT* to *PTPOUT.TXT*, the entire process as described in b) above applies to this command.

It has been occasionally observed that the *PTPOUT.TXT* file will contain a trailing form-feed character after the file has been stripped of all other artifacts. The root cause of this minor issue has been determined to be an obscure quirk of the OS/8 PTP: handler due to an intrinsic design problem within the handler. Notably, if the identical file is processed by the OS/8 LPT: handler [into the *LPTOUT.TXT* file] the problem doesn't occur.

Should this situation be problematic, the character can be removed manually using the *Open PTP Output* command as described in f) below; alternatively, the entire stripping process can be repeated to eliminate the unwanted artifact.

Due to the manner in which the OS/8 PTP: handler functions, while the form-feed character is the last such character in the original file, it is not actually the sentinel character of the file, generally surrounded by sequences of unwanted characters requiring removal at a different stage of the stripping process. Future releases of the *STRIP.TEC* macro may be able to remove this defect in more instances than presently. [As of the current release of the stripping program, this artifact is already being removed in many cases, albeit apparently not in every case.]

- f) *Open PTP Output*. The *PTPOUT.TXT* file is opened in the standard Windows Notepad editor. This allows immediate display of the consequences of processing such as assembling the source file currently under development.

Other than the change of target file from *LPTOUT.TXT* to *PTPOUT.TXT*, the entire process as described in c) above applies to this command.

- g) *Swap PTR and PTP Files*. The contents of the *PTRIN.TXT* [PTR:] and *PTPOUT.TXT* [PTP:] files are interchanged. This provides file interchange useful in specific circumstances.

For example, binary files created in P?S/8 can be made available for OS/8 usage [or vice-versa] using this command. This allows OS/8 users to take advantage of the unique features of P?S/8 *PAL* which supports additional functionality beyond OS/8 *PAL8* while maintaining total *PAL* language compatibility.

With the availability of the P?S/8 *PTUTIL* program, text files may be directly interchanged between these operating systems without involving the host Windows environment. Note: Other methods are available for file interchange between these two operating systems. Appendix V describes a variety of methods, some of which are currently under development.

1.65 Miscellaneous commands

Note: As shown in Figure 3, all miscellaneous commands are located in the bottom row of the Command directory window.

- a) SIMH Working Files. The *SIMH_Working_Files* directory is opened in a separate window. Using Windows Explorer techniques, various files may be accessed for such purposes as file interchange with external sources.

This folder is also the working directory for device image files. By copying device image files here [and editing the appropriate SIMH configuration files] the contents of device image files can be made active in the SIMH environment. Appendix Y details the important files located in this directory.

Note: Normal activity of the various components of the PEPS package will tend to produce temporary files in this directory; in extreme cases, it may be necessary to manually remove temporary files. However, the majority of temporary filenames are internally recycled, largely avoiding any maintenance requirements.

- b) Help Documents. The *SIMH Documents* directory is opened in a separate window, allowing access to a variety of documentation files on various PEPS-related subjects. Since most of the files are in .PDF format, a suitable program such as *Adobe Reader* must be installed. Appendix U provides additional information on the available documents.
- c) Inactive Programs. The *Inactive Programs* directory is opened in a separate window. Generally, icon/link files are stored here for programming projects not currently under development. Several of these files are provided by advanced users of the PEPS package for the purpose of developing their own PDP-8 programs. All of the files are useful as templates for creating additional personal program development commands.

Note: The provided files in the *Inactive Programs* directory will not be actionable without the underlying support files. In some cases, the referenced source files are proprietary and are not available with the standard release of the PEPS package. However, the mechanism to exercise the icon/link files should be functional because dummy files are provided in lieu of the original source files.

- d) SIMH Config Files. The *SIMH Configuration Files* directory is opened in a separate window. Within this directory are the configuration files used with each variant operating system used with the SIMH simulator. Changes made to these files can greatly affect the way the corresponding operating systems function.

References.

1. The *Poly Question Society* (abbreviated wherever possible as P?S, or PQS if only a limited character set is available and pronounced as if spelled pee-kew-ess) is an informal group of questioners concerned with a multitude of subjects. In general, the members of the P?S (whether alive, dead or imaginary) met at an institution of higher learning known as the *Polytechnic Institute of Brooklyn* or more informally "Brooklyn Poly" (now a division of *New York University*). Nearly all P?S members are computer programmers known for exceptional works designed for various computer architectures, especially the DEC PDP-8.

The P?S is an outgrowth of a pre-existing organization known as the *William Claude Fields Memorial Programming Group* (WCFMPG) (also known as *Friends of the Boston Massacre*). The various works of the WCFMPG are legendary in the realm of PDP-8 programming; all original WCFMPG members were responsible for either starting or creating various software projects for DEC computers, especially the PDP-8. All members of the WCFMPG were DEC employees at some point in their careers; however, much of their work on PDP-8 systems (and other computers made by other companies) predates their employment at Digital.

2. The SIMH project is maintained independently and supports many computer architectures. More information on the SIMH project can be obtained from various Internet websites. Appendix D above contains additional information specific to the PDP-8 implementation of SIMH that may be of interest to certain users of the PEPS package; initialization parameters that are important options used in all configurations of the PEPS will be described in greater detail.
3. OS/8 is the principal marketing name of an operating system that can run on most PDP-8 models (or in some instances a specific variant name is used because of the need to support slightly different and likely incompatible models which cannot run the main system components). The group of name variant systems constitutes a confusing mess because some of these variations were merely created for the purposes of highly dubious marketing ploys; only an expert can tell the difference between a variant with little or no technical purpose over OS/8 Version 3D (also known as Version 3Q) and another slightly different system that can only be used with DECmates, albeit not necessarily completely functional). In general it is prudent to avoid these newer variants when using the SIMH simulator due to the likelihood of subtle incompatibilities suddenly appearing seemingly without warning.

Certain specific programs (taken from newer release variants as mentioned above) provide acceptable functionality when used in OS/8 V3D. As such, the image copy of OS/8 provided in the PEPS package contains several notable programs that correct bugs and quirks in the originally released versions associated with OS/8 Version V3D.

OS/8 was originally created as a quick-and-dirty hack pulled off over a single weekend intended as a demo program to be shown to DEC management the next Monday morning. While there was every intention to write a second system with more reasoned design internals, foolishly, management insisted all further changes be made to the demo version instead of a complete rewrite. As a direct result, many serious problems became institutionalized into the design that cannot be corrected.

The original demo (and some of the follow-on work) was done by Richard Lary, who also created most of the R-L Monitor System (also known later as *MS/8*) along with the rest of the members of the WCFMPG. The R-L Monitor System was the first PDP-8 operating system ever used by most of the members of the P?S.

Part of the reason the deficient demo was created is because the R-L Monitor System was shown to DEC management personnel, all of whom rejected it essentially because they had no idea what it was or what it actually did; the demo was designed to show how a computer user could approach the PDP-8 in a manner vaguely similar to the way this was done on other DEC computers available at the time such as using TOPS10 on the PDP-10 from the point of view of a single user. As such, the managers obtained a completely superficial and flawed understanding of the demo (which for them was likely the most they could muster considering their minimal understanding of the other operating systems mentioned above that were in place at the time).

It is noteworthy that all of the other members of the WCFMPG refused to work on the demo because of its flimsy nature. Later, some other members of the P?S were hired by DEC to work on OS/8 along with a few unrelated staffers of varying abilities. While this mixed group did develop the demo into something somewhat better, the inherent internal limitations would never be overcome.

Using OS/8 today, whether on real-world hardware or within the PEPS environment, requires mastering certain quirky skills on the part of the user such as working in a mythical year environment sometime prior to 01-Jan-2002. Unfortunately, due to limitations within the file system, it is not possible to determine the proper year of any particular file because there is an inherent anomaly regarding which group of eight years in the range of 1970-1977, 1978-1985, 1986-1993 or 1994-2001 is the true file origination date reported by the available utilities.

In specific instances, particular users know the proper year group for certain files from personal experience, but inevitably, any collection of files will become commingled with files from one of the other three possible date groups; worse still, program files created today must have mythical dates. Newer programmers must keep track of the real-world date offset on a file by file basis. Because of this serious anomaly, coupled with the advantages offered by advanced features of the PEPS package, it is strongly recommended that users embrace host editing of source files, where more descriptive filenames are possible with accurate file dates.

Certain intrepid users are known to manually insert pragmatic date modifications in the output files of OS/8 directory listings to maintain fantasy records that may contain more information than OS/8 can actually provide. However, due to Y2K considerations, this practice has an upper limit of 31-Dec-2069.

4. P?S/8 is a replacement system for the R-L Monitor system. It includes thousands of improvements borne of all-new work carried out over the course of decades. Despite all of this, the basic system is as low-overhead (or in some minor ways actually lower overhead) as is possible for a small-scale PDP-8 system as compared to the original R-L Monitor System. On many specific hardware configurations, P?S/8 runs palpably faster than OS/8 performing comparable tasks.

Certain specific concepts unique to the R-L Monitor System (and not translatable to OS/8) have either been faithfully implemented in P?S/8, or in some cases, actually improved.

P?S/8 is still being developed. While the P?S/8 release included in the current PEPS package is Version 8Z first released 23-Feb-1988, various improvements and bug fixes have been applied to the system image included in the PEPS release.

As soon as P?S/8 Version 9A is completed, it will replace the present Version 8Z release and be included in future PEPS packages.

P?S/8 runs on all of the same PDP-8 models that OS/8 (and its variants) runs on; however, one P?S/8 release covers all models, differing only in terms of specific device handlers and related utilities where applicable. It is noteworthy that a P?S/8 system based on RX01 diskette media can be booted to all suitably configured models from the original "straight" PDP-8 through the DECmate II. By way of comparison, three different OS/8 variants are required to support the same models due to insurmountable incompatibilities inherent in OS/8 internal design (and at least four were marketed for the purpose). Note: No OS/8 variant exists to support bootable RX01 diskettes on the DECmate II.

A minor patch has to be applied to the supplied P?S/8 release for the TC01/TC08 DECTape controller due to SIMH incompatibility with the real-world hardware with regard to specialized handling of DECTapes with regard to tape rewind and unload operations. On the real-world hardware, performance improvements are readily obtained by appropriate use of the associated P?S/8 keyboard monitor commands. Documentation is available explaining the minor (reversible) patching procedure needed if moving the system image to an actual DECTape for use on real-world systems. Various programs exist to move DECTape images between actual DECTapes and the SIMH-compatible files used in the PEPS environment. Appendix D provides links to appropriate utilities for this and related purposes.

5. BIN format is the standard binary loading format defined in the paper-tape software system. On occasion, a BIN format file must be loaded into memory in the SIMH simulated environment. Both the OS/8 and P?S/8 operating systems support utilities to process BIN format data passed to the SIMH environment by setting up the *PTRIN.TXT* file with the intended binary contents. When files are moved from the SIMH environment to the host windows system, the *PTPOUT.TXT* file is used.

In-progress notes and other junk from here onwards; at this point nearly gone!

<ftp://simh.trailing-edge.com/pub/mirror/www.aracnet.com/~healyzh/pdp8emu.html>

[End-of-file]